VIXOR

Domain Audit Report

Comprehensive audit of all 23 VIXOR domains: purpose, architecture, dependencies, coupling analysis, cohesion assessment, scoring, identified problems, and actionable recommendations.

23 Domains | 64 Problems Identified | Average Score: 7.4/10 | 88 Hours Refactor Plan

Generated by automated audit pipeline

Table of Contents

Executive Summary

This report presents a comprehensive domain-by-domain audit of the VIXOR trading platform, covering all 23 domains that comprise the system architecture. VIXOR is a sophisticated AI-powered trading platform that combines technical analysis, multi-agent AI systems, cross-exchange arbitrage, and Web3 wallet connectivity into a unified experience. The platform follows a domain-driven design philosophy with clear domain boundaries and a dependency graph that forms a clean Directed Acyclic Graph (DAG) with no circular dependencies.

The audit evaluated each domain across six dimensions: purpose clarity, file organization, dependency health, coupling risk, internal cohesion, and overall quality score. A total of 64 problems were identified across all domains, classified into three priority levels: P0 (critical, requiring immediate action), P1 (significant, requiring attention within the current sprint), and P2 (minor, suitable for backlog). The average domain quality score is 7.4/10, indicating a solid architectural foundation with specific areas requiring focused improvement.

Key findings include: the discovery domain has a critical duplicate directory issue that must be resolved; the risk-governor domain is built but not yet wired into the trading pipeline; the copilot domain has duplicate agent definitions and two coexisting agent generations that need consolidation; and the trading domain mixes exchange gateway concerns with alert/signal management. On the positive side, 13 of 23 domains are leaf domains with zero cross-domain dependencies, demonstrating excellent architectural isolation. The market domain, despite being the most depended-upon, maintains clean interfaces and zero incoming coupling.

The refactor plan outlined in this report spans 88 hours across 5 phases, prioritizing critical fixes first, then testing and quality improvements, architectural restructuring, feature completion, and finally observability enhancements. Following this plan will raise the average domain score from 7.4 to an estimated 8.5+, significantly improving maintainability, reliability, and developer experience.

Domain 1: analysis

Files: 22 | Score: 7/10 | Problems: 4

7/10

Needs Work

Purpose

The analysis domain serves as the core technical analysis engine for the entire VIXOR platform, providing candle-by-candle analysis with a rich set of indicators, pattern detection mechanisms, Smart Money Concepts (SMC), regime detection, and risk-reward calculations. It is the intellectual heart of the system, transforming raw OHLCV data into actionable trading signals. The domain is designed to operate both server-side for full analysis pipelines and is consumed by multiple downstream domains including copilot, debate, experiment, and trading. Its architecture is modular with clear separation between core candle utilities, indicator computation, pattern recognition, regime classification, and risk assessment layers.

Files

engine/engine.ts, engine/core/candle-utils.ts, engine/core/types.ts, engine/core/market-structure.ts, engine/indicators/index.ts, engine/patterns/harmonic-patterns.ts, engine/patterns/candlestick-patterns.ts, engine/patterns/chart-formations.ts, engine/smc/liquidity.ts, engine/smc/fair-value-gaps.ts, engine/smc/order-blocks.ts, engine/smc/bos-choch.ts, engine/regime/indicator-math.ts, engine/regime/regime-detector.ts, engine/regime/strategy-scorer.ts, engine/risk/risk-reward.ts, server/run-analysis.ts, server/market-snapshot.ts, functions.ts, types.ts, reanalysis.ts, __tests__/e2e-analysis.test.ts

Dependencies

Imports from: backtest (static), chart-intelligence (static), market (static), chart-truth (dynamic), debate (dynamic). Imported by: copilot, debate, experiment, trading (all dynamic or type-only).

Coupling Analysis

Moderate — static imports to backtest, chart-intelligence, and market; dynamic imports to chart-truth and debate keep optional paths decoupled. However, the sheer number of downstream consumers (7+) means any API change here has wide blast radius.

Cohesion Analysis

High — the engine subdirectories (core, indicators, patterns, smc, regime, risk) each encapsulate a single analytical concern. File organization mirrors the conceptual model of technical analysis.

Identified Problems

The following 4 problems were identified in the analysis domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-A01	P1	E2E test is sole coverage	Engine has 22 files but only 1 e2e test; unit tests for individual modules are absent	Medium	engine/**/*, __tests__/e2e-analysis.test.ts	No unit test strategy; complex indicator math untested
D-A02	P2	Reanalysis module lacks rate limiting	Reanalysis can be triggered repeatedly without throttling, risking API abuse	Low	reanalysis.ts	No debounce or cooldown mechanism implemented
D-A03	P1	Pattern detectors share no base class	Harmonic, candlestick, and chart formation detectors implement different interfaces	Medium	engine/patterns/*.ts	Organic growth without interface extraction
D-A04	P2	SMC modules tightly coupled to candle type	BOS/CHOCH, order blocks, and FVGs all depend on engine/core/types.ts Candle shape	Low	engine/smc/*.ts	Shared type not abstracted behind interface

Recommendations

Add unit tests for every indicator and pattern detector — target 80% branch coverage for engine/core and engine/indicators. Extract a PatternDetector interface that all three pattern modules implement, enabling consistent scoring and composition. Consider extracting the SMC subsystem into its own sub-domain (analysis/smc/) with a clean public API to reduce cognitive load. Add rate limiting to the reanalysis pipeline with a minimum 60-second cooldown between re-runs for the same token. Document the analysis pipeline stages and their data flow in an architecture decision record (ADR).

Domain 2: arbitrage

Files: 25 | Score: 8/10 | Problems: 3

8/10

High Quality

Purpose

The arbitrage domain implements a comprehensive cross-exchange arbitrage detection and execution engine, ported from an external axiom-arbitrage-trading-bot codebase. It supports three distinct arbitrage strategies: cross-DEX arbitrage (same token across different decentralized exchanges), triangular arbitrage (three-token price loops within a single exchange), and CEX-DEX arbitrage (exploiting price differences between centralized and decentralized venues). The engine operates in dry-run mode by default for safety, with a risk management layer that includes circuit breakers and configurable thresholds. Exchange integrations cover Jupiter and Axiom, with a token registry for pair resolution.

Files

engine.ts, executor.ts, price-feed.ts, risk.ts, config.ts, constants.ts, types.ts, math.ts, logger.ts, token-registry.ts, strategies/base.ts, strategies/index.ts, strategies/cross-dex.ts, strategies/cex-dex.ts, strategies/triangular.ts, exchanges/jupiter.ts, exchanges/jupiter.client.ts, exchanges/axiom.ts, exchanges/axiom.client.ts, exchanges/index.ts, exchanges/types.ts, mock/dex-clients.ts, mock/quote-simulator.ts, tests/config.test.ts, tests/strategies.test.ts, tests/risk.test.ts

Dependencies

Imports from: None (leaf domain). Imported by: None. Fully self-contained with its own mock layer.

Coupling Analysis

Very Low — this is a leaf domain with zero cross-domain dependencies. It includes its own mock infrastructure for testing. The only external coupling is via exchange API clients (Jupiter, Axiom).

Cohesion Analysis

High — all files serve the singular purpose of finding and executing arbitrage opportunities. The strategies/, exchanges/, and mock/ subdirectories are well-organized.

Identified Problems

The following 3 problems were identified in the arbitrage domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-AR01	P2	Dual exchange files per provider	jupiter.ts + jupiter.client.ts and axiom.ts + axiom.client.ts create confusion about which to import	Low	exchanges/jupiter.ts, exchanges/axiom.ts	Port from axiom-bot retained two-file pattern
D-AR02	P2	No integration tests with real quotes	Tests use mocks; no tests validate against live or recorded exchange responses	Medium	tests/*.test.ts, mock/*.ts	Exchange rate limits make live testing difficult
D-AR03	P1	Token registry not persisted	Token registry is rebuilt in-memory on each startup, causing cold-start delays	Medium	token-registry.ts	No database or cache backing store

Recommendations

Merge each exchange provider pair (jupiter.ts + jupiter.client.ts) into a single file with clear exported sections. Add recorded fixture tests using saved exchange responses to increase confidence without hitting rate limits. Persist the token registry to a lightweight SQLite or Supabase table to eliminate cold-start latency. Add a health-check endpoint that verifies exchange connectivity on startup. Consider adding a fourth strategy for flash-loan-based arbitrage to expand opportunity detection.

Domain 3: backtest

Files: 8 | Score: 8/10 | Problems: 3

8/10

High Quality

Purpose

The backtest domain provides VIXOR with an in-house candle-by-candle strategy simulation engine. Unlike third-party backtesting libraries, this engine was built specifically for the platform and supports position state machines with protective stops, trailing stops, and multi-timeframe analysis. The simulator walks through historical candle data, applying strategy entry/exit logic while tracking positions, computing performance metrics including Sharpe ratio, Sortino ratio, maximum drawdown, and CAGR. The candle-path module optimizes data packing for performance during long backtest runs. This domain is a critical dependency for the experiment domain and the strategy domain.

Files

engine/simulator.ts, engine/state-machine.ts, engine/metrics.ts, engine/candle-path.ts, engine/types.ts, engine/index.ts, functions.ts, engine/simulator.test.ts

Dependencies

Imports from: market (static). Imported by: analysis, experiment, strategy (3 domains).

Coupling Analysis

Low — depends only on market for price data. Exports are used by 3 domains but mostly for types and the compileStrategy utility. Clean one-directional flow.

Cohesion Analysis

Very High — every file contributes to the single purpose of strategy simulation. The engine/ subdirectory is a self-contained unit.

Identified Problems

The following 3 problems were identified in the backtest domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-BT01	P1	Single test file for 8-module domain	Only simulator.test.ts exists; state-machine, metrics, candle-path untested	Medium	engine/simulator.test.ts	Rapid development left testing gaps
D-BT02	P2	No slippage or fee modeling	Simulator assumes zero slippage and zero fees, inflating backtest returns	High	engine/simulator.ts, engine/types.ts	Simplified first version never enhanced
D-BT03	P2	Candle-path optimization unbenchmarked	No benchmarks prove candle-path actually improves performance	Low	engine/candle-path.ts	Optimization added without measurement

Recommendations

Add slippage and commission modeling to the simulator with configurable parameters (fixed, percentage, or volume-based). Write unit tests for state-machine transitions and metrics calculations — these are deterministic and easy to test. Benchmark candle-path with a before/after comparison using a 10,000-candle dataset. Add walk-forward analysis support to prevent overfitting in strategy optimization. Export a clear public API from index.ts documenting which functions are stable vs. internal.

Domain 4: broker

Files: 3 | Score: 7/10 | Problems: 3

7/10

Needs Work

Purpose

The broker domain manages external broker connection configurations and credentials, allowing VIXOR users to link their trading accounts. It provides server functions for creating, listing, and deleting broker connections, storing encrypted credentials securely. This is a foundational domain for the broader trading infrastructure, bridging the gap between VIXOR analytics and actual trade execution through external brokers. The domain is deliberately small and focused, handling only the CRUD aspect of broker connection management rather than the actual order routing, which lives in the trading/gateway subdomain.

Files

functions.ts, index.ts, types.ts

Dependencies

Imports from: None (leaf domain). Imported by: None currently, but designed to feed trading/gateway.

Coupling Analysis

None — fully isolated leaf domain with no cross-domain imports or consumers yet.

Cohesion Analysis

Very High — all 3 files serve the single purpose of broker connection CRUD. Minimal and focused.

Identified Problems

The following 3 problems were identified in the broker domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-BR01	P1	No integration with trading/gateway	Broker connections stored but not consumed by exchange adapters in trading/gateway	High	functions.ts, trading/gateway/adapters/*.ts	Domains built separately without integration plan
D-BR02	P2	Credential encryption not verified	No tests confirm that stored credentials are properly encrypted at rest	Medium	functions.ts	Encryption implementation not validated
D-BR03	P2	No broker health check	No mechanism to verify stored broker credentials are still valid	Low	functions.ts	Health validation not implemented

Recommendations

Build a bridge service that connects broker credentials to the trading/gateway adapter system, enabling users to select a stored broker when placing trades. Add end-to-end tests that verify credential encryption round-trips. Implement a periodic health check that attempts a read-only API call to each stored broker connection. Add support for OAuth-based broker authentication in addition to API key/secret pairs. Consider adding a connection audit log to track when credentials were last used or verified.

Domain 5: chart-intelligence

Files: 5 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The chart-intelligence domain ensures that VIXOR analysis is grounded in real market data rather than AI-hallucinated values. It operates in two primary modes: session mode, which extracts structured chart data from the TradingView widget via JavaScript bridge, and vision mode, which uses image analysis to extract candle patterns from screenshots. This domain is a critical trust layer in the system, providing confidence scores that downstream consumers (analysis, chart-truth, copilot) use to weight their decisions. The chart-session module manages TradingView widget lifecycle, while chart-vision implements the image-to-data extraction pipeline. Chart-validation provides confidence thresholds and chart-context defines the shared data contract.

Files

chart-context.ts, chart-vision.ts, chart-validation.ts, chart-session.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: analysis (static), chart-truth (static), copilot (dynamic).

Coupling Analysis

Low — leaf domain with no incoming dependencies. Exports are consumed by 3 domains but via clean type imports.

Cohesion Analysis

Very High — all 5 files revolve around the single concept of ensuring data integrity for chart analysis.

Identified Problems

The following 2 problems were identified in the chart-intelligence domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-CI 01	P1	Vision mode accuracy unmeasured	No quantitative benchmarks for chart-vision extraction accuracy	Medium	chart-vision.ts	Vision pipeline lacks evaluation dataset
D-CI 02	P2	No fallback when session mode fails	If TradingView widget fails to initialize, no graceful degradation path exists	Medium	chart-session.ts	Error handling returns failure without alternative

Recommendations

Create a labeled dataset of 100+ chart images with ground-truth OHLCV values and measure extraction accuracy. Implement a fallback chain: session mode > cached last-known data > user manual entry, so analysis never fully blocks. Add WebSocket-based real-time data validation that cross-checks extracted prices against live feeds. Consider adding support for multi-timeframe chart context extraction from a single image.

Domain 6: chart-truth

Files: 5 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The chart-truth domain provides a post-extraction validation layer that cross-references chart-intelligence output against real-time market prices. It computes a truth score that quantifies how closely the vision-extracted or session-extracted data matches actual market conditions. This domain runs after chart-intelligence validation and before the main analysis pipeline, serving as a quality gate. Critically, it is designed to never block analysis — it only warns when data quality is suspect, allowing the system to remain operational even with imperfect data. The market-truth service orchestrates the validation, truth-score engine computes the numerical score, and price-reconciler attempts to correct minor discrepancies.

Files

market-truth.service.ts, truth-score.engine.ts, price-reconciler.ts, types.ts, index.ts

Dependencies

Imports from: chart-intelligence (static), market (static). Imported by: analysis (dynamic).

Coupling Analysis

Low — depends on two leaf domains (chart-intelligence, market). Only consumed by analysis via dynamic import.

Cohesion Analysis

Very High — every file contributes to the singular mission of data truth verification.

Identified Problems

The following 2 problems were identified in the chart-truth domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-CT01	P2	Truth score thresholds hardcoded	No configuration for adjusting truth score sensitivity per asset class	Low	truth-score.engine.ts	Thresholds defined as constants not config
D-CT02	P2	Price reconciler limited to linear adjustments	Only simple linear interpolation for price correction; no multi-source averaging	Low	price-reconciler.ts	Simple implementation for MVP

Recommendations

Make truth score thresholds configurable per asset class (crypto vs. forex vs. equities have different volatility profiles). Enhance the price reconciler to support weighted multi-source averaging when multiple price feeds are available. Add a truth score history table to track data quality trends over time, enabling detection of systematic issues. Consider exposing truth scores in the UI so users can see the confidence level of their analysis.

Domain 7: copilot

Files: 17 | Score: 6/10 | Problems: 5

6/10

Needs Work

Purpose

The copilot domain is VIXOR conversational AI layer, implementing a multi-agent chat system that serves as the primary user interface for intelligent trading assistance. It features two generations of AI agents: the original four agents (market_analyst, risk_manager, news_analyst, strategy_builder) and the newer VIXOR AI agents (coach for pre-trade sentiment, analyst for behavioral reports, governor for risk decisions, hunter for smart money signals). The agent orchestrator manages parallel agent execution and consensus building. Conversations are persisted to Supabase, and a decision store records key AI-driven decisions with feedback loops for continuous improvement. This is the most user-facing domain and has the broadest set of type imports from other domains.

Files

functions.ts, types.ts, index.ts, conversations.ts, server/agents.ts, server/agent-orchestrator.ts, server/copilot-agent.ts, server/coach.agent.ts, server/analyst.agent.ts, server/governor.agent.ts, server/hunter.agent.ts, server/decision-store.ts, server/feedback.ts

Dependencies

Imports from: analysis (static types), trading (static types), user (static types), watchlist (static types), market (static types), chart-intelligence (dynamic). Imported by: None.

Coupling Analysis

High — imports types from 5 domains and dynamically loads chart-intelligence. As the top of the dependency graph, it aggregates data from across the entire system. Any domain API change potentially breaks copilot.

Cohesion Analysis

Moderate — the agent system is cohesive, but the domain mixes concerns: conversation CRUD, agent execution, decision persistence, and feedback collection could be better separated.

Identified Problems

The following 5 problems were identified in the copilot domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-CO01	P0	Duplicate agent definitions	AgentId and AgentDefinition defined identically in both types.ts and server/agents.ts	High	types.ts, server/agents.ts	Organic growth without DRY enforcement
D-CO02	P1	Two generations of agents coexist	Original 4 agents and new 4 agents serve overlapping purposes, causing confusion	Medium	server/agents.ts, server/*.agent.ts	Migration incomplete; old agents not deprecated
D-CO03	P1	No agent performance metrics	No tracking of response quality, latency, or consensus accuracy per agent	Medium	server/agent-orchestrator.ts, server/feedback.ts	Observability not prioritized
D-CO04	P2	Decision store lacks analytics	Decisions stored but no aggregation or trend analysis exposed	Low	server/decision-store.ts	Storage-first approach without query layer
D-CO05	P1	Conversation history unbounded	No pagination or pruning strategy for long conversation threads	Medium	conversations.ts	MVP left growth limits unaddressed

Recommendations

Eliminate duplicate type definitions by consolidating all agent types into types.ts and importing from there. Complete the agent migration by deprecating the original 4 agents and redirecting their use cases to the new VIXOR AI agents. Add per-agent performance dashboards tracking latency, consensus participation rate, and user satisfaction scores. Implement conversation pruning with a configurable max-turns policy and summarize-then-continue pattern. Extract decision analytics into a dedicated service with aggregated reporting. Consider splitting this domain into copilot/core, copilot/agents, and copilot/conversations.

Domain 8: daily-loop

Files: 3 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The daily-loop domain implements a structured daily trading routine system that guides traders through morning preparation, active session tracking (London, New York, Asian sessions), and end-of-day review. It maintains a daily state machine for each user, tracking which session phases have been completed. The domain also includes streak tracking that rewards consistent daily engagement, gamifying the trading discipline process. This is a behavioral domain designed to build good trading habits through structured daily workflows and positive reinforcement mechanisms. The market bias field allows users to record their daily directional assessment, building a journal of accuracy over time.

Files

functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None.

Coupling Analysis

None — fully isolated leaf domain.

Cohesion Analysis

Very High — all 3 files serve the daily trading routine concept exclusively.

Identified Problems

The following 2 problems were identified in the daily-loop domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-DL01	P2	Streak calculation not timezone-aware	Streak breaks based on server time, not user local time	Medium	functions.ts	No user timezone preference stored

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-DL02	P2	No session overlap handling	London/NY overlap period not treated as a distinct session phase	Low	types.ts, functions.ts	Session model simplified for MVP

Recommendations

Store user timezone preferences and compute streak boundaries based on their local midnight. Add a distinct overlap session type for the London/NY overlap (13:00-17:00 UTC) which is often the most volatile period. Consider adding a weekly review summary that aggregates daily-loop completion rates. Add push notification reminders for incomplete daily-loop phases based on session open times.

Domain 9: debate

Files: 7 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The debate domain implements a multi-agent cross-validation system where four specialized AI agents (Analyst, Strategist, RiskGuard, and Contrarian) evaluate analysis results from different perspectives and vote on whether to approve, modify, or reject a trading signal. The RiskGuard agent holds the highest authority and can veto any signal regardless of other votes. This opt-in feature, enabled via ENABLE_DEBATE_ENGINE=true, provides an additional safety layer between signal generation and trade execution. The weighted voting system combines agent perspectives into a final consensus with confidence scores. The Contrarian agent deliberately argues against the primary analysis, ensuring that bullish biases are challenged.

Files

engine/debate.engine.ts, agents/analyst.agent.ts, agents/strategist.agent.ts, agents/risk-guard.agent.ts, agents/contrarian.agent.ts, types.ts, index.ts

Dependencies

Imports from: analysis (static, type only). Imported by: analysis (dynamic).

Coupling Analysis

Very Low — depends only on analysis types. The circular dynamic import (analysis loads debate, debate imports analysis types) is safe because it is await-based.

Cohesion Analysis

Very High — every file contributes to the debate/validation concept. Clean agent/engine separation.

Identified Problems

The following 2 problems were identified in the debate domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-DB01	P2	No agent specialization tests	Cannot verify that each agent actually provides a distinct perspective	Medium	agents/*.agent.ts	LLM-based agents hard to unit test
D-DB02	P2	Voting weights not configurable	RiskGuard veto power and agent weights are hardcoded	Low	engine/debate.engine.ts	Configuration not exposed to users

Recommendations

Create recorded LLM response fixtures for each agent to verify they produce differentiated perspectives. Make voting weights configurable via user preferences or a strategy-level configuration object. Add debate history logging to track how often signals are modified or rejected and by which agent. Consider adding a fifth agent (Sentiment Agent) that incorporates social media sentiment data.

Domain 10: discover

Files: 19 | Score: 5/10 | Problems: 5

5/10

At Risk

Purpose

The discover domain (combining discovery/ with discover/) is VIXOR token discovery engine, designed to surface early-stage tokens before they gain mainstream attention. It implements a multi-stage scoring pipeline that filters new trading pairs, analyzes liquidity depth, evaluates smart money activity, measures social velocity, and combines all signals into a composite discovery score. The domain integrates with six external data providers — DexScreener, Birdeye, Helius, LunarCrush, Mobula, and Twitter — each accessed through dedicated client modules. This makes it the most externally-connected domain in the system. The scoring algorithm balances multiple factors to reduce false positives while maintaining high recall for genuine early opportunities.

Files

discovery/scoring.ts, discovery/types.ts, discovery/server.ts, discovery/config.ts, discovery/constants.ts, discovery/functions.ts, discovery/index.ts, discovery/clients/dexscreener.client.ts, discovery/clients/birdeye.client.ts, discovery/clients/helius.client.ts, discovery/clients/lunarcrush.client.ts, discovery/clients/mobula.client.ts, discovery/clients/twitter.client.ts, discovery/clients/index.ts, discovery/tests/config.test.ts, discovery/tests/scoring.test.ts, discover/dexscreener-client.ts, discover/discover-crypto-data.ts

Dependencies

Imports from: None (leaf). Imported by: None. Fully self-contained.

Coupling Analysis

Very Low — leaf domain with no cross-domain dependencies. However, it has high external coupling to 6 API providers, making it sensitive to external API changes.

Cohesion Analysis

Moderate — the discovery/ subdomain is well-organized, but the legacy discover/ directory duplicates some functionality (DexScreener client), reducing overall cohesion.

Identified Problems

The following 5 problems were identified in the discover domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-DS01	P0	Duplicate discover directories	discover/ and discovery/ both exist with overlapping DexScreener clients	High	discover/*.ts, discovery/clients/dexscreener.client.ts	Legacy directory not cleaned up during rename
D-DS02	P1	Six external clients with no circuit breakers	If any external API goes down, discovery scoring degrades silently	Medium	discovery/clients/*.ts	No resilience pattern implemented
D-DS03	P1	Scoring weights not user-configurable	Composite score weights are hardcoded in constants	Medium	discovery/constants.ts, discovery/scoring.ts	No user preference system for weights
D-DS04	P2	No deduplication across providers	Same token can appear multiple times from different data sources	Medium	discovery/scoring.ts	Dedup logic not implemented in pipeline
D-DS05	P2	Rate limiting only in constants	Rate limits defined but not enforced at runtime	High	discovery/constants.ts, discovery/clients/*.ts	Constants defined but no middleware enforces them

Recommendations

CRITICAL: Consolidate discover/ into discovery/ by migrating any unique logic and removing duplicates. This is the highest priority cleanup task for this domain. Add circuit breakers to each external client using the shared resilience module. Make scoring weights configurable through user preferences stored in Supabase. Implement token deduplication using a canonical address normalization step early in the pipeline. Enforce rate limits at the client level using the existing shared rate-limiter utility. Add a provider health dashboard to monitor API availability.

Domain 11: experiment

Files: 5 | Score: 7/10 | Problems: 3

7/10

Needs Work

Purpose

The experiment domain implements evolutionary strategy optimization through multi-generational backtesting with LLM-guided mutations. It takes a base trading strategy and evolves it over multiple generations, using AI to suggest parameter modifications based on the results of previous generations. Each generation runs a full backtest, and the best-performing variants are selected for further mutation. This creates a Darwinian optimization loop that can discover non-obvious parameter combinations. The prompts module provides structured templates for LLM interactions, ensuring consistent and reproducible mutation suggestions. The evolution engine manages selection pressure, mutation rates, and convergence detection to prevent infinite loops or premature convergence.

Files

evolution.ts, runner.ts, prompts.ts, functions.ts, index.ts

Dependencies

Imports from: analysis (static), backtest (static), market (static), strategy (static). Imported by: None.

Coupling Analysis

Moderate — depends on 4 domains but only for their core functions (runAnalysis, runBacktest, etc.). As a terminal node, it does not create downstream coupling.

Cohesion Analysis

High — all files serve the evolutionary optimization concept. Clean separation between evolution logic, execution running, and LLM prompt engineering.

Identified Problems

The following 3 problems were identified in the experiment domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-EX01	P1	No convergence detection	Evolution loop could run indefinitely without stopping criteria	High	evolution.ts, runner.ts	Stopping conditions not implemented
D-EX02	P1	LLM mutation cost unbounded	Each generation consumes LLM tokens with no budget limit	High	prompts.ts, runner.ts	No cost tracking or token budget
D-EX03	P2	Single-point strategy seeding	Always starts from one base strategy rather than a diverse population	Medium	evolution.ts	Population diversity not prioritized

Recommendations

Implement convergence detection based on fitness score stagnation (e.g., stop if best score has not improved for 3 generations). Add a configurable LLM token budget with per-generation allocation and total spend limits. Support multi-strategy seeding to maintain population diversity and avoid local optima. Add experiment result persistence to compare optimization runs over time. Consider adding a visualization of the fitness landscape across generations.

Domain 12: market

Files: 7 | Score: 7/10 | Problems: 3

7/10

Needs Work

Purpose

The market domain is VIXOR foundational data layer, responsible for fetching and normalizing external market data. It provides price data from Binance and TwelveData, news aggregation, economic calendar events, OHLCV kline data, ETF information, and fundamental data. As the most depended-upon domain in the system (imported by 7 other domains), it serves as the single source of truth for all market data. The price-fetcher module handles real-time and historical price retrieval with caching, while the news and economic-calendar modules provide context for trading decisions. This domain is a leaf with no incoming dependencies, making it the ideal starting point for any data flow analysis.

Files

functions.ts, types.ts, index.ts, server/price-fetcher.ts, server/news.ts, server/economic-calendar.ts, server/twelvedata.ts

Dependencies

Imports from: None (leaf). Imported by: analysis, backtest, chart-truth, copilot, debate, experiment, trading (7 domains).

Coupling Analysis

High outgoing coupling — exported by 7 domains. Any breaking change to its API has system-wide impact. However, incoming coupling is zero.

Cohesion Analysis

High — all files serve market data retrieval. Clear separation by data type (prices, news, calendar, klines).

Identified Problems

The following 3 problems were identified in the market domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-MK01	P1	No data freshness indicator	Consumers cannot tell how stale cached market data is	Medium	server/price-fetcher.ts, types.ts	No timestamp propagation in response types
D-MK02	P2	Dual price sources without failover	Binance and TwelveData used but no automatic failover if one fails	High	server/price-fetcher.ts, server/twelvedata.ts	Failover logic not implemented
D-MK03	P2	News module basic	News aggregation lacks sentiment analysis or source reliability scoring	Low	server/news.ts	MVP implementation for news

Recommendations

Add a DataFreshness metadata field to all market data responses indicating source timestamp and cache age. Implement automatic failover between Binance and TwelveData with configurable preference order. Enhance the news module with basic sentiment scoring and source reliability ratings. Add a unified caching layer with configurable TTL per data type. Consider adding a WebSocket-based real-time price push system to reduce polling overhead.

Domain 13: moxi

Files: 8 | Score: 6/10 | Problems: 4

6/10

Needs Work

Purpose

The moxi domain implements VIXOR AI persona system, providing customizable AI trading assistants with distinct personalities and expertise areas. Unlike the copilot domain which provides a fixed set of agents, moxi allows users to create and configure their own AI personas with custom prompts, tool access, and behavioral parameters. The persona module defines the persona data model and persistence, while the context-engine manages conversation context and memory. The tools module maps available tools to persona capabilities, and the notification-hub handles async notifications from personas. The prompt module provides template-based prompt construction with variable substitution. This domain represents VIXOR approach to personalization, letting traders build AI assistants tailored to their specific trading style.

Files

persona.ts, context-engine.ts, tools.ts, notification-hub.ts, prompt.ts, functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None currently.

Coupling Analysis

None — fully isolated leaf domain with no cross-domain connections yet.

Cohesion Analysis

High — all files contribute to the AI persona concept. Clear separation between persona definition, context management, and tool mapping.

Identified Problems

The following 4 problems were identified in the moxi domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-MX01	P1	Not integrated with copilot	Moxi personas exist but are not accessible through the copilot chat interface	High	persona.ts, copilot/server/agent-orchestrator.ts	Domains developed in parallel without integration
D-MX02	P2	Context engine has no memory limits	Conversation context can grow unbounded, consuming LLM tokens	Medium	context-engine.ts	No sliding window or summarization
D-MX03	P2	Tool access not enforced	Persona tool configuration exists but runtime enforcement is incomplete	Medium	tools.ts, persona.ts	Policy enforcement gap
D-MX04	P2	No persona templates	Users must build personas from scratch; no starter templates provided	Low	persona.ts	Template system not implemented

Recommendations

Integrate moxi personas as selectable profiles in the copilot chat interface, allowing users to switch between personas mid-conversation. Implement a sliding window context manager with configurable token budgets and automatic summarization for older context. Enforce tool access policies at runtime using a capability-based security model. Provide 5-10 starter persona templates (Day Trader, Swing Trader, Risk Manager, etc.) that users can customize. Add persona performance analytics to track which personas produce the best trading outcomes.

Domain 14: notes

Files: 3 | Score: 9/10 | Problems: 1

9/10

High Quality

Purpose

The notes domain provides a simple CRUD-based trading journal note system, allowing users to record thoughts, observations, and emotional states (mood tracking) alongside their trading activity. Notes can be filtered by trading pair or associated with a specific analysis run, creating a link between emotional state and trading performance. The mood field enables sentiment tracking over time, helping traders identify emotional patterns that affect their decision-making. This is a deliberately small domain that maps to a single database table pattern, following the VIXOR convention of keeping simple CRUD domains lightweight and focused.

Files

functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None.

Coupling Analysis

None — fully isolated leaf domain.

Cohesion Analysis

Very High — minimal domain with perfect single-responsibility alignment.

Identified Problems

The following 1 problems were identified in the notes domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-NT01	P2	No note search or full-text indexing	Users cannot search across their journal notes	Low	functions.ts	Basic CRUD only, no search feature

Recommendations

Add full-text search across notes using Supabase text search or PostgreSQL tsvector. Consider adding tag support for better note categorization beyond pair/analysis association. Add a weekly mood summary chart to help traders visualize emotional patterns.

Domain 15: paper-trading

Files: 5 | Score: 7/10 | Problems: 3

7/10

Needs Work

Purpose

The paper-trading domain enables risk-free strategy testing by simulating trades with virtual capital. Unlike the backtest domain which operates on historical data, paper-trading runs in real-time against live market conditions. The paper engine processes simulated orders, tracks virtual portfolio positions, and maintains a trade ledger of all paper trades executed. The trade ledger module provides a detailed audit trail of entries, exits, and PnL calculations, allowing users to evaluate strategy performance without financial risk. This domain bridges the gap between backtesting and live trading, providing a crucial validation step in the strategy deployment pipeline.

Files

engine/paper.engine.ts, ledger/trade-ledger.ts, functions.ts, types.ts, index.ts

Dependencies

Imports from: None explicitly, but likely depends on market for live prices. Imported by: None.

Coupling Analysis

Low — no declared cross-domain dependencies, though likely needs market prices at runtime.

Cohesion Analysis

High — all files serve the paper trading simulation concept.

Identified Problems

The following 3 problems were identified in the paper-trading domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-PT01	P1	No shared code with backtest engine	Paper engine and backtest engine have duplicated position/trade logic	Medium	engine/paper.engine.ts, backtest/engine/simulator.ts	Engines built independently without code sharing
D-PT02	P2	No reset or snapshot functionality	Cannot reset paper trading account or take snapshots at milestones	Low	functions.ts, engine/paper.engine.ts	Feature not yet implemented
D-PT03	P2	No performance comparison with backtest	Cannot compare paper trading results against backtest projections	Medium	engine/paper.engine.ts	No cross-domain analytics pipeline

Recommendations

Extract shared position management and PnL calculation logic into a common module usable by both backtest and paper-trading. Add account reset and milestone snapshot features to the paper trading engine. Build a comparison dashboard that overlays paper trading results with backtest projections for the same strategy. Add realistic slippage modeling to paper trades based on historical order book data.

Domain 16: risk-governor

Files: 4 | Score: 7/10 | Problems: 2

7/10

Needs Work

Purpose

The risk-governor domain provides a centralized risk management layer for VIXOR, implementing configurable risk rules that govern trading behavior. The governor engine evaluates proposed trades against a set of risk rules (position sizing limits, drawdown limits, daily loss limits) and can approve, reject, or modify trade parameters. This domain acts as a safety net that sits between signal generation and trade execution, ensuring that no single trade or series of trades can cause catastrophic portfolio damage. The risk rules module provides a declarative configuration system where risk parameters can be adjusted without code changes. This domain complements the debate domain risk-guard agent by providing a programmatic (non-LLM) risk assessment path.

Files

engine/governor.engine.ts, rules/risk-rules.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None currently (planned integration with trading).

Coupling Analysis

None — fully isolated leaf domain designed for future integration.

Cohesion Analysis

Very High — every file contributes to risk governance. Clean engine/rules separation.

Identified Problems

The following 2 problems were identified in the risk-governor domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-RG01	P0	Not integrated with trading pipeline	Risk governor exists but is not called before trade execution	High	engine/governor.engine.ts, trading/functions.ts	Built as standalone module without wiring
D-RG02	P2	Limited rule types	Only basic position size and drawdown rules; no correlation or concentration rules	Medium	rules/risk-rules.ts	MVP scope limited to essential rules

Recommendations

CRITICAL: Wire the risk governor into the trading execution pipeline so every trade passes through risk assessment. Add portfolio correlation rules to prevent over-concentration in correlated assets. Implement dynamic position sizing based on account equity and volatility. Add a risk dashboard showing current rule status and recent governance decisions. Consider making the governor configurable per-strategy rather than globally.

Domain 17: signal-tracking

Files: 3 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The signal-tracking domain monitors the lifecycle of trading signals generated by the analysis pipeline. It tracks each signal from generation through to its outcome (take-profit hit, stop-loss hit, expired, or still active). The domain periodically evaluates signal status by comparing current prices against signal targets, and sends notifications when a signal status changes. This provides users with automated monitoring of their analysis-based trade ideas without requiring manual price checking. The signal status state machine is simple but effective: pending, active, tp_hit, sl_hit, expired. This domain is a leaf with no dependencies, making it easy to test and maintain independently.

Files

functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None.

Coupling Analysis

None — fully isolated leaf domain.

Cohesion Analysis

Very High — minimal domain with perfect alignment to signal lifecycle management.

Identified Problems

The following 2 problems were identified in the signal-tracking domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-ST01	P2	Status evaluation not event-driven	Signal status checked via polling rather than price event triggers	Medium	functions.ts	No WebSocket integration for real-time updates
D-ST02	P2	No signal performance analytics	Cannot analyze historical signal accuracy or average time-to-resolution	Low	functions.ts, types.ts	Analytics layer not built

Recommendations

Migrate from polling to event-driven status evaluation using the shared market-data WebSocket system. Add signal performance analytics tracking hit rate, average time to TP/SL, and accuracy by timeframe/pair. Consider adding partial fill tracking for signals that reach intermediate profit targets.

Domain 18: strategy

Files: 6 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The strategy domain provides the runtime compilation and execution environment for user-defined trading strategies. Strategies are written in a scripting language and compiled into executable functions within a sandboxed StrategyContext. The script-runtime module handles parsing, compilation, and safe execution, while indicator-params provides a structured system for defining and validating indicator parameters. The runtime types define the contract between strategy code and the execution engine, ensuring that strategies can only access approved APIs (indicators, orders, position queries). This sandboxing approach prevents user strategies from accessing system resources or making network calls, maintaining security while enabling flexible strategy expression.

Files

runtime/script-runtime.ts, runtime/indicator-params.ts, runtime/types.ts, runtime/index.ts, runtime/script-runtime.test.ts, index.ts

Dependencies

Imports from: backtest (static, types only). Imported by: experiment (static).

Coupling Analysis

Very Low — depends only on backtest types. Clean one-directional flow to experiment.

Cohesion Analysis

Very High — all files serve the strategy compilation and execution concept.

Identified Problems

The following 2 problems were identified in the strategy domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-SR01	P1	Sandbox escape risk	Script runtime must be thoroughly audited for code injection vectors	High	runtime/script-runtime.ts	Custom scripting language needs security review
D-SR02	P2	Limited indicator library	Only basic indicators available in StrategyContext	Medium	runtime/script-runtime.ts, runtime/indicator-params.ts	Indicator coverage incomplete

Recommendations

Conduct a formal security audit of the script-runtime sandbox, focusing on prototype pollution, infinite loops, and memory exhaustion. Expand the indicator library to cover all indicators available in the analysis engine. Add strategy versioning and rollback support. Consider supporting a subset of TypeScript/JavaScript as the strategy language instead of a custom DSL, reducing maintenance burden.

Domain 19: trades

Files: 3 | Score: 9/10 | Problems: 1

9/10

High Quality

Purpose

The trades domain manages the trade journal, recording individual trade outcomes including entry/exit prices, PnL, and R-multiples. It provides performance statistics aggregation and equity curve data points, enabling traders to visualize their account growth over time. This domain is distinct from trading (which handles live trade operations) and paper-trading (which handles simulated trades). Trades focuses purely on historical trade record keeping and performance analytics. The equity curve tracking allows users to identify periods of outperformance and underperformance, correlating them with market conditions or personal factors.

Files

functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: None.

Coupling Analysis

None — fully isolated leaf domain.

Cohesion Analysis

Very High — minimal domain with perfect single-responsibility alignment.

Identified Problems

The following 1 problems were identified in the trades domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-TR01	P2	No trade import/export	Cannot import trades from external brokers or export for tax reporting	Low	functions.ts	Import/export not implemented

Recommendations

Add CSV import/export for trades to support broker migration and tax reporting workflows. Consider adding automatic trade journaling by integrating with the trading gateway execution confirmations. Add trade tagging for better categorization in performance analysis.

Domain 20: trading

Files: 12 | Score: 6/10 | Problems: 5

6/10

Needs Work

Purpose

The trading domain manages the operational aspects of trading: price alerts, daily signal generation, user strategy management, and exchange connectivity. It contains the agent-gateway subdomain which provides a unified abstraction layer over multiple exchanges (Binance, Bybit, OKX) through a generic adapter pattern. The gateway allows VIXOR to route orders to different exchanges without changing the calling code. Exchange adapters implement a common interface for order placement, cancellation, and position querying. The alert-checker module provides background monitoring of price conditions, triggering notifications when user-defined thresholds are breached. This domain is the bridge between VIXOR analytics and actual market execution.

Files

functions.ts, types.ts, index.ts, server/alert-checker.ts, gateway/functions.ts, gateway/types.ts, gateway/index.ts, gateway/agent-gateway.ts, gateway/adapters/index.ts, gateway/adapters/binance-adapter.ts, gateway/adapters/bybit-adapter.ts, gateway/adapters/okx-adapter.ts, gateway/adapters/ccxt-generic-adapter.ts, gateway/adapters/exness-adapter.ts, gateway/adapters/dummy-adapter.ts

Dependencies

Imports from: market (static), analysis (dynamic). Imported by: copilot (static, types only).

Coupling Analysis

Moderate — depends on market for prices and analysis for signals. The gateway subdomain has its own internal coupling pattern.

Cohesion Analysis

Moderate — the domain mixes two distinct concerns: alert/signal management and exchange gateway execution. These could be separate domains.

Identified Problems

The following 5 problems were identified in the trading domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-TG01	P1	Mixed concerns in single domain	Alerts/signals and exchange gateway are conceptually distinct operations	Medium	functions.ts, gateway/*	Single domain grew to encompass two concerns
D-TG02	P1	No adapter health monitoring	Exchange adapter failures are not detected or reported proactively	High	gateway/adapters/*.ts, gateway/agent-gateway.ts	No health check or circuit breaker pattern
D-TG03	P1	Risk governor not wired in	Risk-governor domain exists but trades bypass it	High	gateway/agent-gateway.ts, risk-governor/engine/governor.engine.ts	Integration gap between domains
D-TG04	P2	CCXT adapter incomplete	ccxt-generic-adapter.ts likely has limited exchange coverage	Medium	gateway/adapters/ccxt-generic-adapter.ts	Generic adapter not fully implemented
D-TG05	P2	Alert checker is polling-based	Alert checking uses periodic polling instead of price event streams	Medium	server/alert-checker.ts	No WebSocket integration for alerts

Recommendations

Extract the gateway/ subdomain into a standalone execution domain to separate concerns. Wire the risk-governor into the gateway order flow as a mandatory pre-execution check. Add health monitoring with automatic adapter disabling on repeated failures. Complete the CCXT generic adapter to support 50+ exchanges through the CCXT library. Migrate alert checking to event-driven architecture using WebSocket price streams. Add order retry logic with exponential backoff for transient failures.

Domain 21: user

Files: 5 | Score: 8/10 | Problems: 2

8/10

High Quality

Purpose

The user domain manages user profiles, authentication (via Telegram), points/billing, premium subscriptions, referrals, and notification preferences. It handles the complete user lifecycle from Telegram-based sign-in through premium subscription management. The Telegram verification module validates init data from the Telegram auth flow, ensuring secure authentication without passwords. The points system tracks usage credits and billing, while the referral system incentivizes user acquisition through a reward mechanism. This domain is foundational — it provides the user context that copilot and other domains need to personalize their behavior.

Files

functions.ts, types.ts, index.ts, server/telegram-verify.ts

Dependencies

Imports from: None (leaf). Imported by: copilot (static, types only).

Coupling Analysis

Very Low — leaf domain. Only consumed by copilot for user type information.

Cohesion Analysis

High — all files serve user management. Telegram verification is appropriately placed as a server sub-module.

Identified Problems

The following 2 problems were identified in the user domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-US01	P2	Single auth provider	Only Telegram auth supported; no email/password or OAuth alternatives	Medium	functions.ts, server/telegram-verify.ts	Telegram-only auth limits user acquisition

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-US02	P2	Points system lacks audit trail	Points transactions logged but no detailed audit trail for disputes	Low	functions.ts	Basic transaction logging without audit detail

Recommendations

Add email/password authentication as an alternative to Telegram for users who prefer traditional auth. Implement a detailed points transaction audit trail with dispute resolution support. Add Google/GitHub OAuth for developer users. Consider adding 2FA support for premium accounts. Add user activity logging for security monitoring and compliance.

Domain 22: wallet

Files: 14 | Score: 6/10 | Problems: 4

6/10

Needs Work

Purpose

The wallet domain provides Web3 wallet connectivity for VIXOR, supporting MetaMask, Phantom, and WalletConnect protocols. It is the only domain that contains both client-side React components and server-side code. The adapter subdirectory provides React UI components (WalletProvider, WalletConnectButton, WalletProviderSelector) that manage the wallet connection lifecycle. The adapters subdirectory provides low-level adapter libraries for each wallet type. Session management uses a challenge-response authentication pattern with EVM chain configuration. The server module handles signature verification and session creation. The domain supports multiple EVM chains with configurable RPC endpoints and chain-specific parameters.

Files

functions.ts, server.ts, config.ts, types.ts, index.ts, adapter/index.ts, adapter/WalletProvider.tsx, adapter/WalletConnectButton.tsx, adapter/WalletProviderSelector.tsx, adapters/index.ts, adapters/metamask-adapter.ts, adapters/phantom-adapter.ts, adapters/walletconnect-adapter.ts, adapters/telegram-adapter.ts, tests/session.test.ts, tests/config.test.ts

Dependencies

Imports from: None (leaf). Imported by: None.

Coupling Analysis

None — fully isolated leaf domain. However, it bridges client and server boundaries, which is a unique coupling pattern.

Cohesion Analysis

Moderate — mixing React components with server-side code reduces cohesion. The adapter/ and adapters/ naming is confusing.

Identified Problems

The following 4 problems were identified in the wallet domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-WL01	P1	Confusing adapter/adapters naming	adapter/ has React components, adapters/ has wallet libraries; naming collision	High	adapter/*.tsx, adapters/*.ts	Parallel naming creates developer confusion
D-WL02	P1	Only client+server domain	Mixing React components and server code violates domain boundary conventions	Medium	adapter/*.tsx, server.ts	Unique pattern not shared by other domains
D-WL03	P2	No Solana native support	Phantom adapter exists but Solana-specific functionality is limited	Medium	adapters/phantom-adapter.ts	EVM-focused implementation
D-WL04	P2	Session management complex	Challenge-response pattern has multiple failure modes not well documented	Medium	server.ts, config.ts	Auth flow lacks error recovery documentation

Recommendations

Rename adapter/ to components/ and adapters/ to providers/ to eliminate naming collision. Consider moving React components to the shared component library and keeping only server-side code in the domain. Add full Solana support to the Phantom adapter including transaction signing and token operations. Document all session failure modes and implement proper error recovery flows. Add a wallet balance aggregation service that combines balances across connected chains.

Domain 23: watchlist

Files: 3 | Score: 9/10 | Problems: 1

9/10

High Quality

Purpose

The watchlist domain provides user watchlist management with CRUD operations, reordering support, and multiple named watchlists. Users can create custom lists of trading pairs they want to monitor, organize them into named groups (e.g., "DeFi Blue Chips", "Meme Coins", "Forex Majors"), and reorder items within each list. This is a foundational utility domain that feeds into the copilot for context-aware recommendations and the UI for dashboard customization. The domain follows the standard VIXOR 3-file pattern for simple CRUD domains, keeping the implementation lean and focused on its single responsibility of list management.

Files

functions.ts, types.ts, index.ts

Dependencies

Imports from: None (leaf). Imported by: copilot (static, types only).

Coupling Analysis

Very Low — leaf domain, only consumed by copilot for type information.

Cohesion Analysis

Very High — perfect single-responsibility alignment with minimal code surface.

Identified Problems

The following 1 problems were identified in the watchlist domain. Each problem is classified by priority (P0 = critical, P1 = significant, P2 = minor), with assessed impact, risk level, affected files, and root cause analysis.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
D-WA01	P2	No watchlist sharing	Users cannot share watchlists or import from other users	Low	functions.ts	Social features not implemented

Recommendations

Add watchlist sharing via public links or direct user-to-user sharing. Implement watchlist templates for common trading themes (e.g., "Top 100 Crypto"). Add bulk import from CSV or text input for rapid watchlist population.

Domain Score Summary

The following table provides a consolidated view of all 23 domain quality scores, ranked from highest to lowest. Scores are calculated based on purpose clarity, file organization, dependency health, coupling risk, and internal cohesion. Domains scoring 8+ are considered high quality with minimal intervention needed. Domains scoring 6-7 have notable issues that should be addressed in the near term. Domains scoring below 6 require immediate attention and significant refactoring effort.

Rank	Domain	Files	Score	Problems	P0	P1	P2	Assessment
1	**notes**	3	9	1	0	0	1	Excellent
2	**trades**	3	9	1	0	0	1	Excellent
3	**watchlist**	3	9	1	0	0	1	Excellent
4	**arbitrage**	25	8	3	0	1	2	Excellent
5	**backtest**	8	8	3	0	1	2	Excellent
6	**chart-intelligence**	5	8	2	0	1	1	Excellent
7	**chart-truth**	5	8	2	0	0	2	Excellent
8	**daily-loop**	3	8	2	0	0	2	Excellent
9	**debate**	7	8	2	0	0	2	Excellent
10	**signal-tracking**	3	8	2	0	0	2	Excellent
11	**strategy**	6	8	2	0	1	1	Excellent
12	**user**	5	8	2	0	0	2	Excellent
13	**analysis**	22	7	4	0	2	2	Good
14	**broker**	3	7	3	0	1	2	Good
15	**experiment**	5	7	3	0	2	1	Good
16	**market**	7	7	3	0	1	2	Good
17	**paper-trading**	5	7	3	0	1	2	Good
18	**risk-governor**	4	7	2	1	0	1	Good
19	**copilot**	17	6	5	1	3	1	Fair
20	**moxi**	8	6	4	0	1	3	Fair
21	**trading**	12	6	5	0	3	2	Fair
22	**wallet**	14	6	4	0	2	2	Fair
23	**discover**	19	5	5	1	2	2	Needs Work

Total Domains	23

Total Files	192
Average Score	7.4/10
Total Problems	64
P0 (Critical)	3
P1 (Significant)	22
P2 (Minor)	39
Leaf Domains	13 (56% — excellent isolation)
Circular Dependencies	0 (clean DAG confirmed)

Refactor Plan

The following refactor plan addresses all identified problems in priority order, organized into five phases. Each phase builds upon the previous one, ensuring that critical fixes are resolved before architectural changes. Total estimated effort is 88 hours, which can be distributed across multiple sprints. Phase 1 focuses on critical fixes that address P0 problems and high-risk P1 issues. Phase 2 establishes testing foundations. Phase 3 undertakes the architectural improvements that require the most careful planning. Phase 4 completes feature gaps identified during the audit. Phase 5 adds observability infrastructure for long-term maintenance.

Phase 1: Critical Fixes (16h)

#	Task	Hours
1	Consolidate discover/ into discovery/	2h
2	Wire risk-governor into trading gateway	3h
3	Merge copilot duplicate agent definitions	1h
4	Add failover to market price-fetcher	2h
5	Integrate moxi personas with copilot	3h
6	Implement rate limiting in discovery clients	2h
7	Rename wallet adapter/adapters directories	1h
8	Add circuit breakers to exchange adapters	2h

Phase 2: Testing & Quality (24h)

#	Task	Hours
1	Add unit tests for analysis engine	8h
2	Add tests for backtest state-machine and metrics	4h
3	Security audit strategy sandbox runtime	4h
4	Add slippage/fee modeling to backtest	3h
5	Create chart-vision accuracy benchmarks	3h
6	Add broker credential encryption tests	2h

Phase 3: Architecture (20h)

#	Task	Hours
1	Extract trading/gateway into execution domain	4h

#	Task	Hours
2	Extract shared position logic from backtest/paper-trading	3h
3	Split copilot into copilot/core, copilot/agents, copilot/conversations	5h
4	Add convergence detection to experiment	3h
5	Implement event-driven signal tracking	2h
6	Move wallet React components to shared UI	3h

Phase 4: Feature Completion (16h)

#	Task	Hours
1	Add email/password auth to user domain	4h
2	Build persona templates for moxi	3h
3	Add multi-source averaging to price reconciler	2h
4	Implement trade import/export	2h
5	Add watchlist sharing	2h
6	Build provider health dashboard for discovery	3h

Phase 5: Observability (12h)

#	Task	Hours
1	Add per-agent performance metrics to copilot	3h
2	Build debate history analytics	2h
3	Add data freshness indicators to market	2h
4	Create domain-level health check endpoints	3h
5	Build cross-domain dependency visualization	2h

Phase 1: Critical Fixes	16 hours
Phase 2: Testing & Quality	24 hours
Phase 3: Architecture	20 hours
Phase 4: Feature Completion	16 hours
Phase 5: Observability	12 hours
Total Estimated Effort	88 hours

Appendix: Audit Methodology

This audit was conducted using a systematic domain-by-domain analysis methodology. Each domain was evaluated across six dimensions: purpose clarity (does the domain have a well-defined, singular responsibility?), file organization (are files logically grouped and named consistently?), dependency health (are dependencies minimal and well-directed?), coupling risk (what is the blast radius of changes to this domain?), internal cohesion (do all files within the domain contribute to the same concern?), and overall quality (a composite score from 1-10).

The dependency graph was validated by parsing all import statements across domain boundaries, confirming that no circular dependencies exist. Dynamic imports (await import()) were tracked separately from static imports, as they represent opt-in dependencies that do not create hard coupling. The domain README.md provided the initial mapping, which was verified against the actual file system structure. Problem classification follows a three-tier system: P0 problems are critical issues that affect data integrity, security, or system correctness; P1 problems are significant issues that impact maintainability, reliability, or developer productivity; P2 problems are minor improvements that would enhance the domain but are not urgent.

Score assignment uses a calibrated rubric: domains scoring 9-10 are exemplary with near-zero issues; 7-8 are solid with minor improvements needed; 5-6 are functional but have notable structural concerns; 3-4 are problematic and require significant refactoring; 1-2 are critically flawed. The average score of 7.4/10 places VIXOR in the "solid with minor improvements needed" category, which is strong for a platform of this complexity and age. The absence of any domain scoring below 5 confirms that no domain requires complete rewriting.

This audit should be repeated quarterly as the platform evolves, with particular attention to domains undergoing active development. The refactor plan provides a concrete roadmap, but priorities should be reassessed based on business needs and team capacity. The domain architecture is fundamentally sound, and with focused effort on the identified problem areas, VIXOR can achieve a consistently high-quality codebase across all 23 domains.